

IMAGE PROCESSING & COMPUTER VISION
NUMPY & OPENCV & INTRODUCING TO IMAGE PROCESSING APPLICATIONS



KHEN MUHAMMAD CAHYO

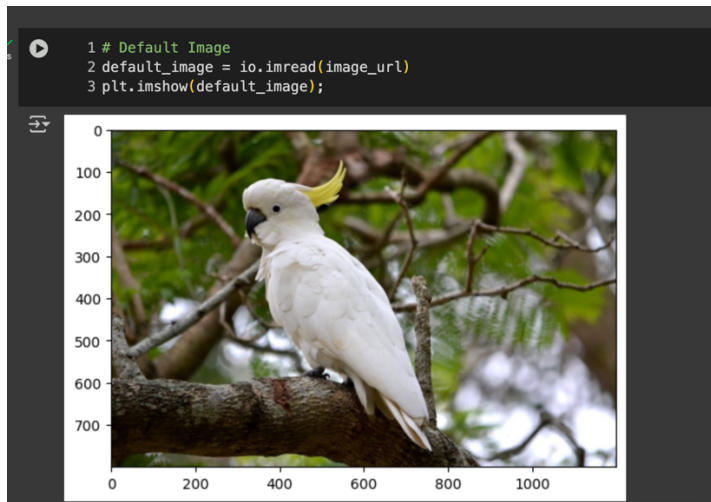
244107023002

TI-3I

STATE POLYTECHNIC OF MALANG
INFORMATION TECHNOLOGY DEPARTMENT

PRACTICUM – IMAGE PROCESSING

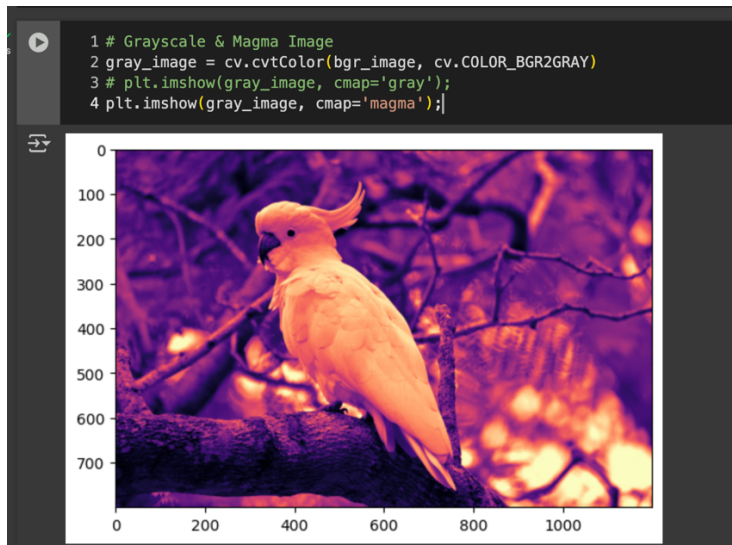
- **Default Image**



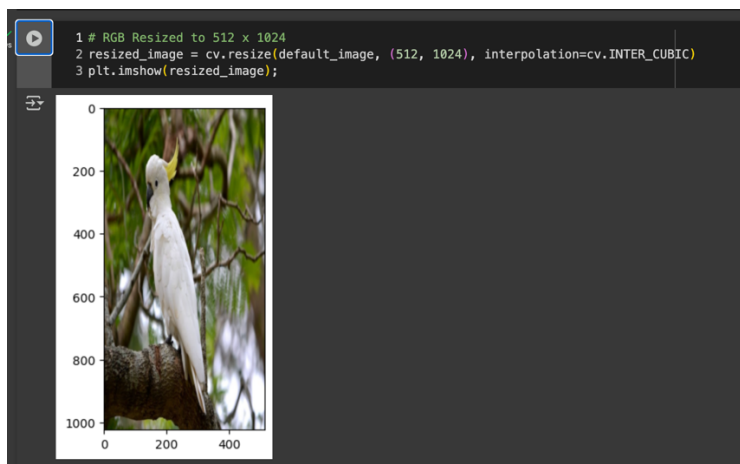
- **Convert to BGR**



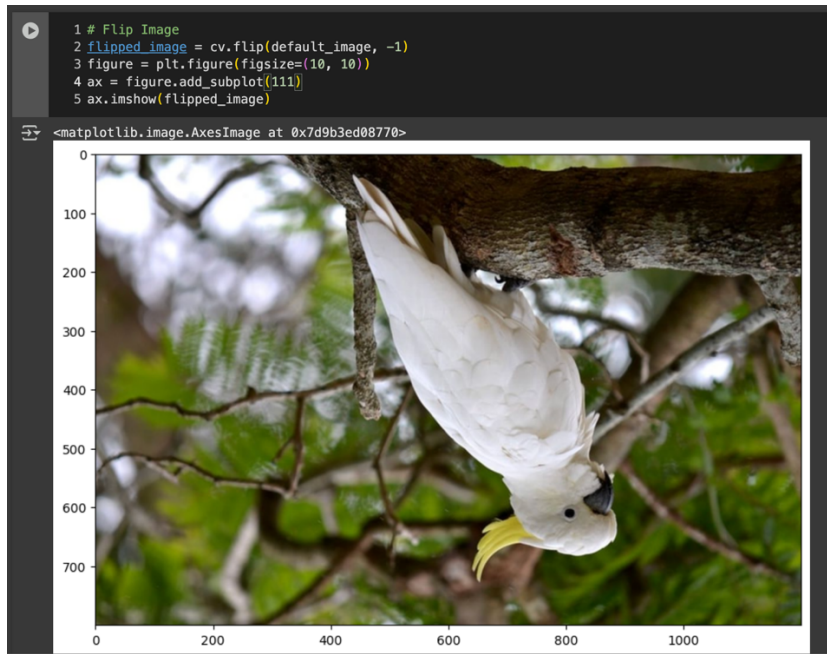
- **Convert to Grayscale & Magma**



- **Resize**

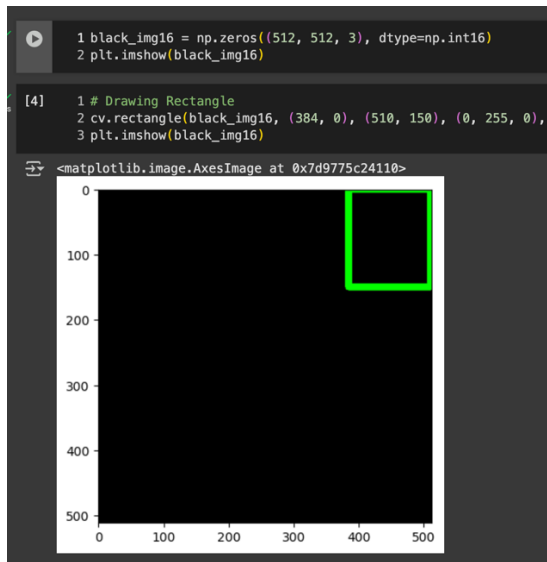


- **Flip**

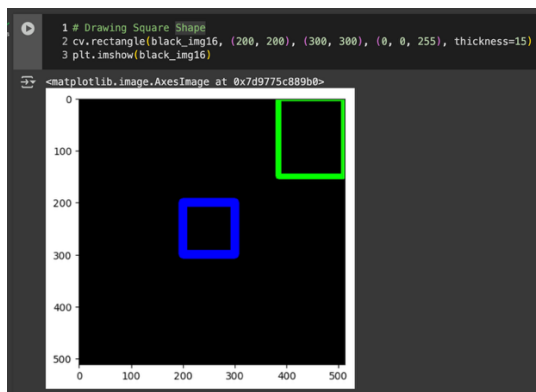


PRACTICUM – BLACK IMAGE (INT16)

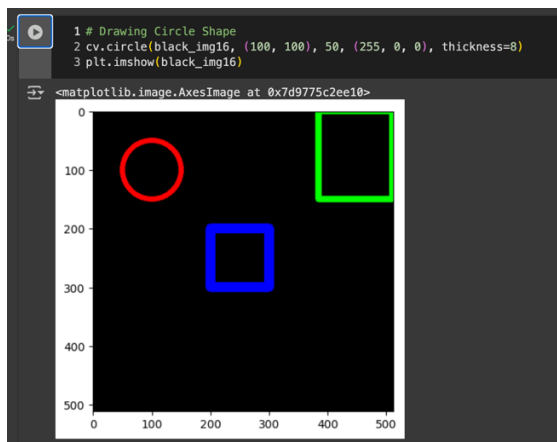
- **Drawing Rectangle**



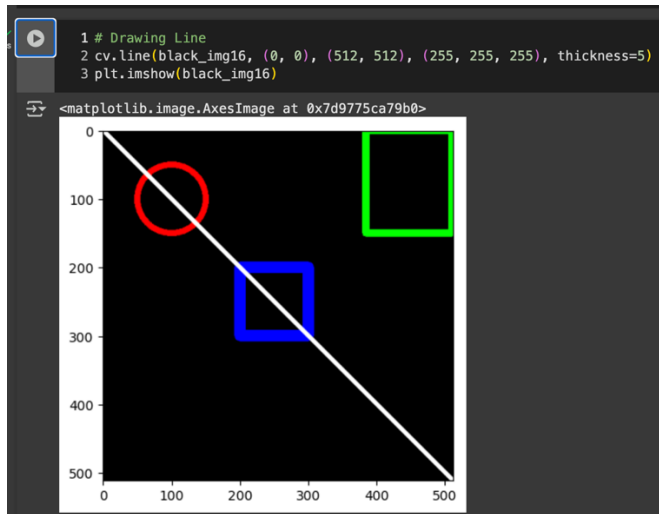
- **Drawing Square Shape**



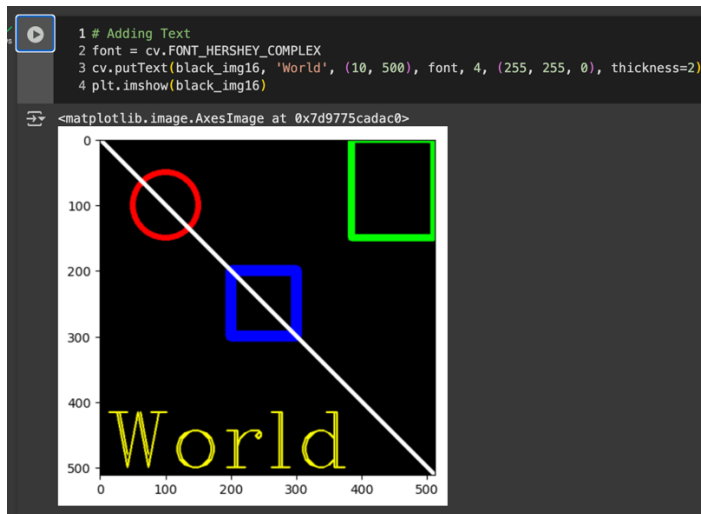
- **Drawing Circle Shape**



- **Drawing Line**

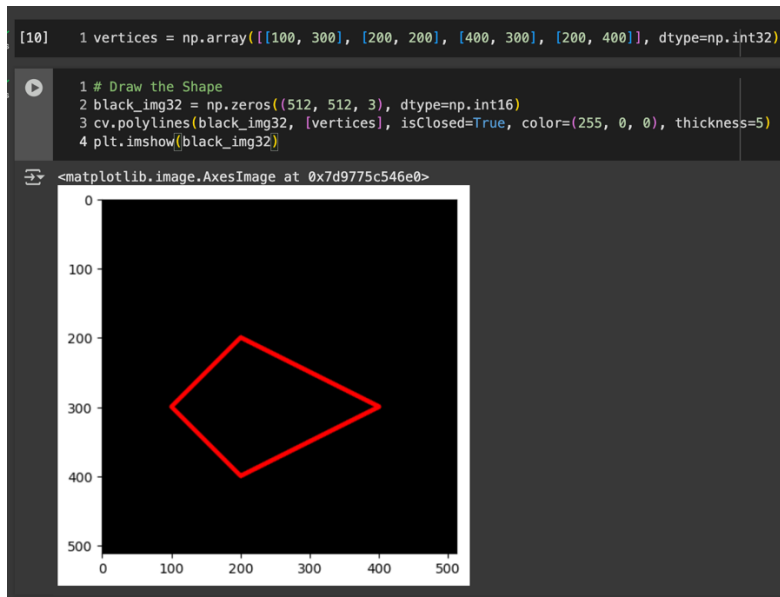


- **Adding Text**



PRACTICUM – BLACK IMAGE (INT32)

- Draw the Shape



QUESTION

Answer Number 1

When we display an image with OpenCV (**cv2.imshow**), it shows the raw pixel values as-is (usually **uint8** in the range 0–255) and interprets colors in BGR order, so it won't normalize or rescale unusual data types like **int16**. In contrast, Matplotlib (**plt.imshow**) expects either floats in [0.0, 1.0] or **uint8** in [0, 255], interprets colors in RGB order, and automatically normalizes other datatypes (like **int16**) to fit into a displayable range, which can cause unexpected colors or brightness. That's why the same array can look different between the two libraries, and for consistency, it's best to convert images to **uint8** before displaying.

Answer Number 2

Using **int16** or **int32** to create a black image with **np.zeros** makes no visible difference in Matplotlib since both will display as black (all zeros), but the real difference lies in their value ranges: **int16** can only represent **–32,768** to **32,767**, while **int32** can store much larger values; this usually doesn't matter for images, which are typically **uint8** (0–255). However, when working with geometry in OpenCV (like polygon vertices), the data type matters, OpenCV requires **int32** for coordinates, because **int16** can overflow for pixel positions larger than **32,767** or simply be rejected by functions like **cv2.fillPoly**, so **int32** is the safe and correct choice for drawing shapes.

Answer Number 3

cv2_imshow from **google.colab.patches** is a helper function provided specifically for Google Colab to display images that are loaded or processed with OpenCV, because the standard **cv2.imshow** does not work in Colab's notebook environment (it relies on a local GUI window, which Colab doesn't support). Instead, **cv2_imshow** converts the image into a format compatible with IPython's display system and shows it directly inside the notebook cell output. In short, we use **cv2_imshow** in Colab whenever we want to visualize an OpenCV image, while in a normal local Python environment we would use **cv2.imshow**.

Answer Number 4

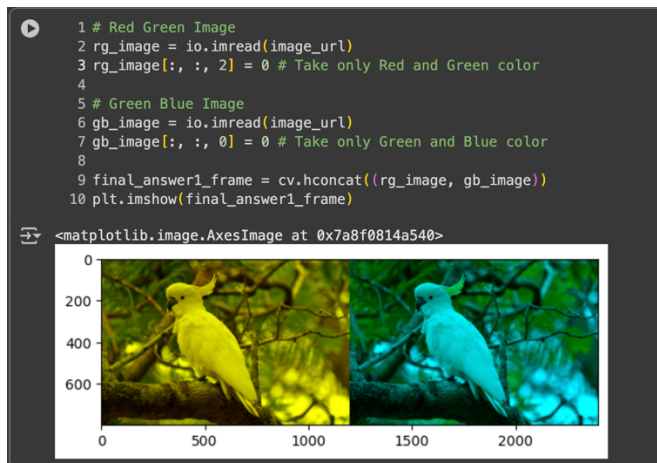
skimage.io (imported as `from skimage import io`) is part of the scikit-image library and provides convenient functions for reading, displaying, and saving images in Python. With it, we can easily load images from files or URLs into NumPy arrays (**`io.imread`**), visualize them inside notebooks or GUIs (**`io.imshow`**), and save processed images back to disk (**`io.imwrite`**). Unlike OpenCV, which defaults to BGR format, **skimage.io** reads images in RGB by default and integrates well with scientific Python tools, making it especially useful for research, prototyping, and image analysis workflows.

TASK

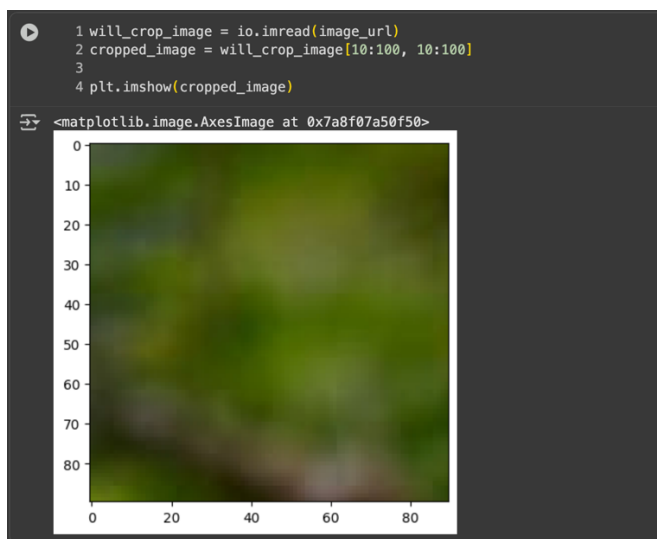
Answer Number 1

When we use **figsize** in Matplotlib, we are only changing the **size of the figure canvas in inches**, not the actual pixel dimensions of the image data itself. The underlying image array (e.g., 512×512 pixels) does not change, it's just displayed larger or smaller on the plotting canvas. The actual rendered pixel size on screen depends on both figsize and the figure's DPI (dots per inch). So, if figsize=(8, 6) with dpi=100, the output will be 800×600 display pixels, but the image still contains the same 512×512 pixel values internally.

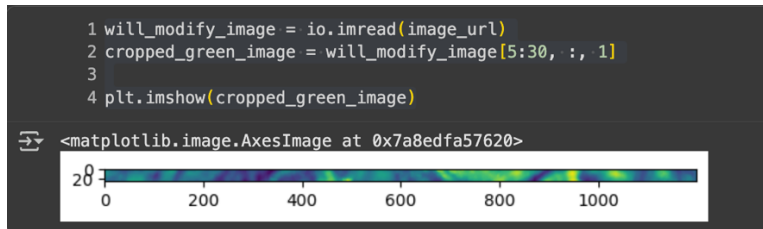
Answer Number 2



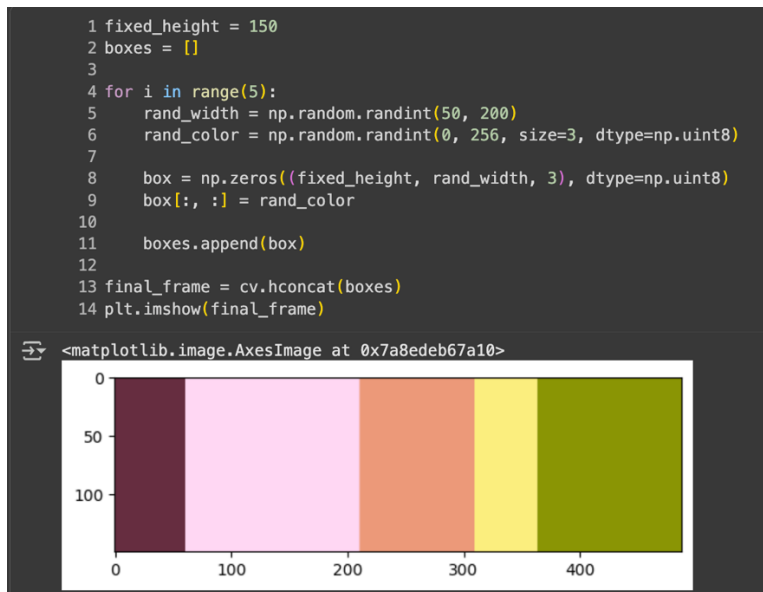
Answer Number 3



Answer Number 4



Answer Number 5



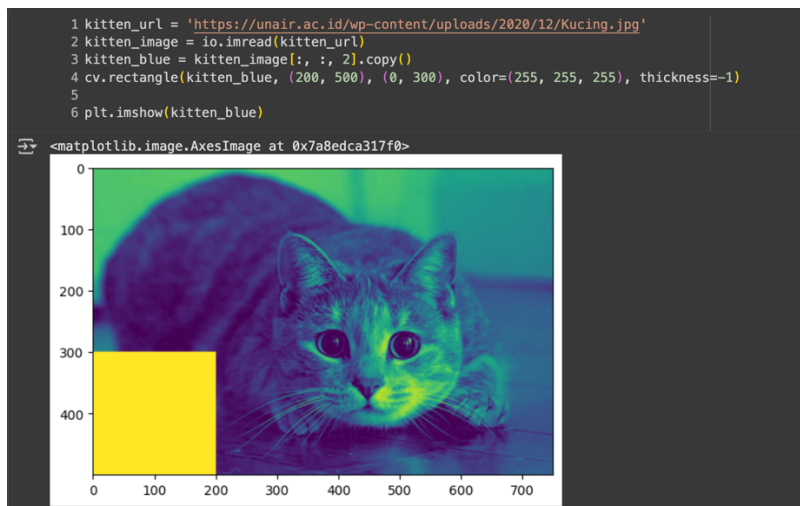
Answer Number 6



Answer Number 7



Answer Number 8

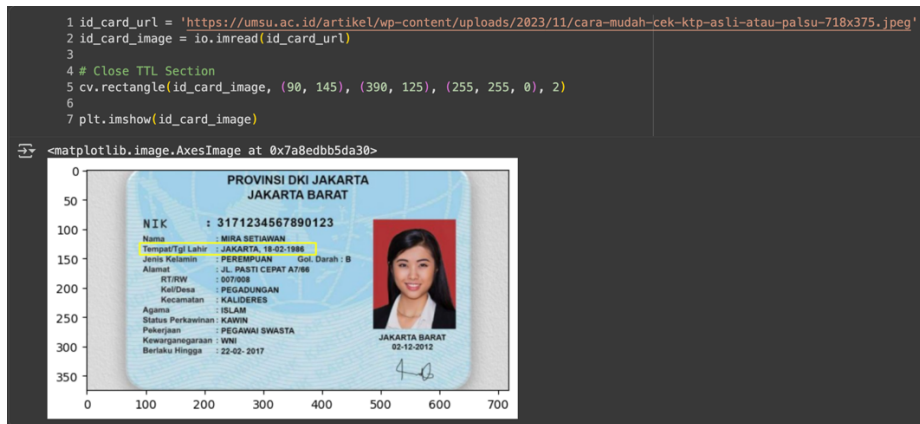


Answer Number 9

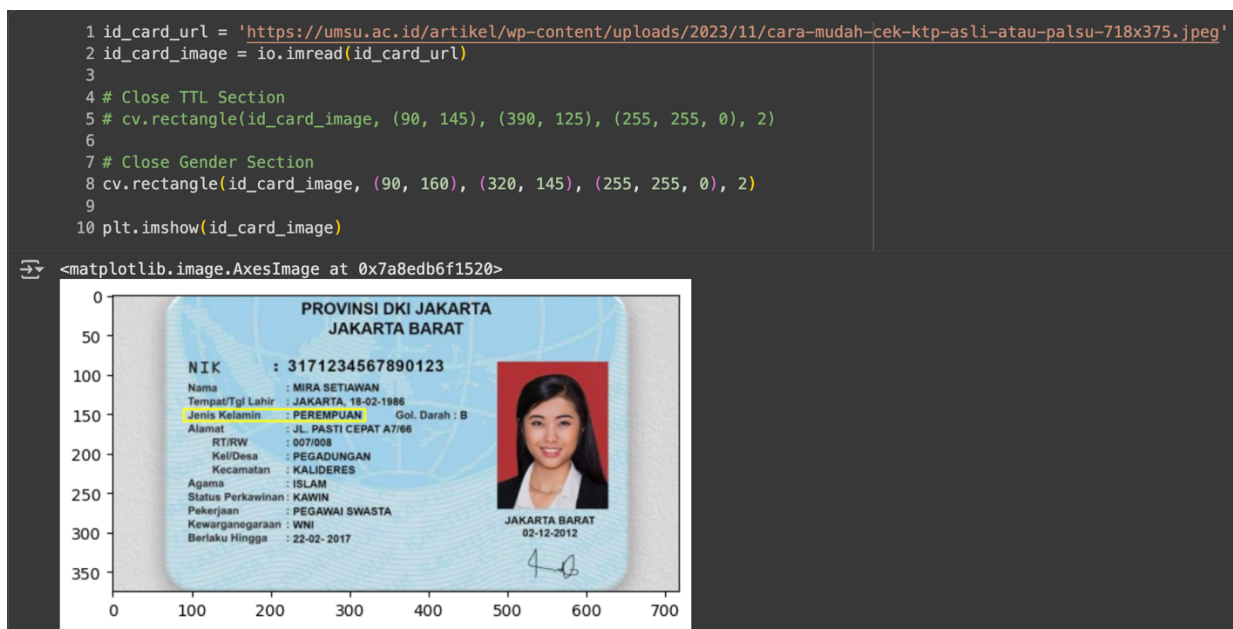


TASK ID CARD

Close TTL Section



Close Gender Section



Github Link

<https://github.com/KhenCahyo13/MyCampus-Jobsheet/tree/smt-5/pcvk/week-2>